PROJECT REPORT · AUGUST 2026

Investment Risk Profiler

An India-only equity intelligence platform: six-factor risk scoring, a three-model machine-learning ensemble that predicts performance *relative to the Nifty 50*, a transformer-based news sentiment pipeline, a retrieval-grounded geopolitical War Room, and significance-tested sector seasonality.

Scope	Everything built to date, how each part works, what is novel, and what to build next
Repository	investment-risk-profiler · branch `feature/india-model-and-war-room`
Stack	Flask (Python 3.11) · React 18 · scikit-learn + XGBoost · FinBERT · Gemini · yfinance
Codebase	~11,400 lines across 45 source files (Python backend, React frontend, JSON knowledge base)
Universe	150 NSE-listed equities trained; 173,000 labelled samples; 13 tracked conflicts; 10 sector indices

Educational analysis system. Nothing in this document or the software is financial advice.

Contents

01 **Executive summary**
What the system is and what it does

02 **What has been built**
Feature inventory and development chronology

03 **Technology stack**
Platform, libraries, data sources, models, APIs

04 **System architecture**
Request lifecycle and module topology

05 **The six-factor risk engine**
Scoring, weighting and dynamic reweighting

06 **Feature deep dives: analysis factors**
Technical, fundamental, sentiment, volumetric, sector, geopolitical

07 **The machine-learning core**
Target design, features, tri-model ensemble, calibration, consensus

08 **Decision-support layers**
Portfolio optimiser, merits, peers, AI insight

09 **The War Room**
Retrieval-grounded geopolitical impact analysis

10 **Seasonal analysis**
Market-relative, significance-tested sector rotation

11 **Trading strategies and backtester**
Quantitative strategy engine

12 **Validation and honest measurement**
Out-of-sample tests and what they show

13 **Frontend and user experience**
Single-page app, service layer, design system

14 **Novelty: what is genuinely different**
Seven defensible design decisions

15 **Limitations and caveats**
What the system cannot do

16 **Roadmap: what to build next**
Near-term, medium-term and ambitious

17 **Reference**
API surface, repository layout, how to run

HOW TO READ THIS DOCUMENT

Sections 01-04 are the overview and the stack. Sections 05-13 are the feature-by-feature deep dives: each one states the problem it addresses, the reasoning behind the design, and the technical implementation. Sections 14-16 cover novelty, honest limitations and the forward roadmap.

SECTION 01

Executive summary

One system, five intelligence layers, one deliberately narrow market

The Investment Risk Profiler is a full-stack analytics platform for Indian listed equities. A user types an NSE ticker, states a risk appetite and an investment horizon, and receives a single consolidated read on that stock: a 0-100 multi-factor risk score with a transparent breakdown, a machine-learning prediction of whether the stock will beat the Nifty 50 over the next month, a transformer-scored view of news sentiment against its own sector, an explicit map of the geopolitical conflicts that touch it, a personalised asset allocation, a plain-language merits-versus-demerits panel, better-performing sector peers if it is lagging, and a narrative synthesis written by a large language model constrained to the computed evidence.

Two further modules sit alongside the per-stock analysis. The **War Room** traces a geopolitical conflict through India's bilateral relations and commodity trade exposure down to named NSE tickers that benefit or suffer. The **Seasonal** module tests whether Indian sector-rotation patterns are statistically real rather than artefacts of overall market drift.

Four design commitments distinguish it

1. **India-only, by construction.** Every ticker is forced onto an Indian exchange, the model is trained purely on NSE price history, the benchmark is the Nifty 50, the macro context is India VIX and USD/INR, and the language model is prompted as a Mumbai sell-side analyst reasoning in the Indian macro frame. A US symbol simply fails validation.
2. **The prediction target is relative, not absolute.** The models answer "will this beat the index?" rather than "will this go up?". Market drift is subtracted out, forcing the model to learn stock-specific edge. This is the single most consequential design decision in the project and it is what produced the first measurable predictive edge over baseline.
3. **Honesty over impressiveness.** Synthetic training data was deleted. Features that could not be reconstructed historically were removed from the model rather than faked. Seasonal patterns are surfaced only when they pass a significance test. Out-of-sample backtests report economic edge, not flattering in-sample accuracy. When the models disagree, the interface says so.
4. **Deterministic first, generative last.** Every number is computed by explicit, auditable code. The language model is the final narration layer and is explicitly forbidden from inventing figures, so a model failure degrades the prose, never the analysis.

Metric	Value	Metric	Value
Source files	45	Training samples	173,000
Lines of code	~11,400	Model ROC-AUC	0.811
Analysis factors	6	Best cross-val accuracy	0.626 vs 0.516 baseline
ML models in ensemble	3	Tracked conflicts	13
Engineered features	26	Mapped NSE tickers in KB	118
NSE tickers trained	150	Sector indices analysed	10

Figure 1. System at a glance. Model metrics are from the current binary relative-performance model retrained on 2026-08-06.

SECTION 02

What has been built

Feature inventory, with the development chronology that produced it

Feature inventory

Module	What it does	Status
Multi-factor risk engine	Six independent analysers scored 0-100 and combined into one weighted risk score with a visible breakdown and dynamic reweighting	Live
Technical analysis	Around 25 indicator families: RSI, MACD, Bollinger, stochastics, VWAP, Heiken-Ashi, Parabolic SAR, Supertrend, Fibonacci, supply/demand zones, volume profile, divergence, harmonic and candlestick patterns, Gann angles	Live
Fundamental analysis	Profitability, efficiency, liquidity, leverage and valuation ratio blocks, plus a qualitative governance and red-flag layer driven by news scanning	Live
Enhanced sentiment	Three-phase pipeline: hybrid RSS and scraped news ingestion, FinBERT transformer scoring, recency- and relevance-weighted aggregation with a stock-versus-sector divergence signal	Live
Volumetric analysis	On-balance volume, price-volume trend, accumulation/distribution, volume regime and confirmation patterns, fully vectorised	Live
Sector comparison	Relative performance against the correct Indian sector index and the Nifty, resolved from the exchange suffix	Live
Geopolitical exposure factor	Deterministic reverse lookup from ticker to every tracked conflict that touches it, weighted by conflict status and India-relevance	Live
Tri-model ML ensemble	Calibrated Random Forest, XGBoost and a rule-aware histogram gradient booster; soft-vote ensemble, explicit consensus reporting, per-model driver importances	Live
Portfolio optimiser	Sizes the equity sleeve from the model's edge over the index and the stock's quality, then caps and shapes it by the investor's stated appetite and horizon; Herfindahl diversification score	Live
Merits and demerits	Deterministic why-to-buy versus why-not panel framed around what different investor types actually care about	Live
Peer alternatives	Ranks the same-sector NSE cohort by six-month return and surfaces better performers as one-click re-analysis chips when the stock lags	Live
Gemini insight layer	India-aware narrative synthesis over the full computed picture, grounded and non-directive	Live
War Room	Retrieval-grounded conflict-to-sector-to-ticker impact reports, from a curated watchlist or a free-text scenario, with click-through into full analysis	Live
Seasonal analysis	Market-relative excess returns per sector-month, t-tested for significance, tagged with the real Indian calendar driver; dual heatmaps	Live
Trading strategies	Trend following, cointegration-tested pairs mean reversion, technical indicator and calendar spread strategies, with a full backtester and risk manager	Built, API only
Validation harnesses	Time-split out-of-sample model backtest and an in-sample/out-of-sample seasonal-edge persistence test	Built, CLI

Development chronology

The project's shape came from a sequence of deliberate revisions, several of which threw away working code because it was measurably dishonest or ineffective. That history is worth recording because it is the argument for the current design.

Stage	Change	Why it mattered
Baseline	Flask plus React risk-profiling questionnaire and first stock analysis modules	Established the multi-factor scaffold and the API contract
Sentiment rebuild	Keyword sentiment replaced by the three-phase FinBERT pipeline with a sector-divergence signal	Headline keyword counting cannot read financial nuance; a sector baseline makes the stock's own mood interpretable
India lock-in	US anchor tickers removed from training, tickers force-normalised to NSE, Gemini prompt rewritten in the Indian macro frame	A model trained on mixed markets learns neither well; narrowing the domain raised quality everywhere
Synthetic data deleted	Rule-generated training rows removed with no fallback; training now fails loudly if real data cannot be fetched	Synthetic rows taught the model the rules it was supposed to be independent of
Train/serve unification	A single feature builder became the only source of truth for both training and inference; fundamentals and sentiment dropped from the model	Fundamentals were constant placeholders in training but real at inference - a silent train/serve mismatch. Free data exposes only about five quarters of fundamentals, so they could not be reconstructed honestly
Feature expansion	12 to 24 price-derived features, then 26 with market-regime context	Cross-validated accuracy rose from 0.445 to 0.515
Third model added	A rule-aware histogram gradient booster trained on the features plus the rule engine's own net score	Gives the ensemble a member that explicitly blends learned patterns with the hand-built rule system
Absolute market features removed	Nifty direction, VIX level and USD/INR dropped after they became the top drivers	Identical across every stock on a given date, they homogenised all predictions toward the market's direction and destroyed stock-to-stock differentiation. Only relative strength and the VIX regime were kept
Horizon lengthened	Label horizon moved from 5 to 20 trading days	Five-day returns are close to noise; twenty days carries trend
Target changed to relative	Three-class absolute sell/hold/buy replaced by binary outperform versus underperform against the Nifty 50	**The decisive change.** The old target was dominated by market drift, so the models were largely re-predicting the index and accuracy sat at the majority baseline. The relative target produced 0.626 cross-validated accuracy against a 0.516 baseline and 0.811 ROC-AUC - the first genuine edge
Seasonality corrected	Raw monthly averages replaced by excess returns against the Nifty plus one-sample t-tests	Raw returns made nearly every sector peak in April - that is market beta, not sector seasonality
Decision layers	Merits panel, peer alternatives, Herfindahl diversification, prescriptive profile guidance	Turned a scoring tool into something a retail investor can reason with

Figure 2. Development chronology. Each row is a change that altered the system's behaviour or its honesty, not merely its surface.

SECTION 03

Technology stack

Platform, libraries, data sources, models and external services

Platform and runtime

Layer	Choice	Notes and rationale
Operating system	Windows 11	Development and deployment target. Several design choices are Windows accommodations, documented below
Backend language	Python 3.11 (global interpreter)	Runs on the global interpreter rather than the project virtual environment, which lacks a consistent dependency set
Web framework	Flask 2.3.3 with Werkzeug 2.3.7 pinned	Flask 2.3.x breaks against Werkzeug 3.x, so the pin is mandatory
Cross-origin	flask-cors 4.0.0	Development frontend on port 3000 talks to the API on port 5000
Frontend	React 18.2 via Create React App 5.0.1	Single-page app, three views, no router; a development proxy forwards API calls to Flask
HTTP client	Axios 1.14	Timeout raised to 150 seconds because a cold analysis with a fresh FinBERT load is slow
Production server	gunicorn (declared, not yet deployed)	Currently runs under the Flask development server
Configuration	python-dotenv	API keys and the model identifier come from environment variables

Data, numerics and machine learning

Library	Version	Role in the system
pandas	2.2.x	All time-series handling; upper-bounded because pandas 3.x risks breaking yfinance
numpy	1.26.x	Vectorised feature computation, held below 2.0 for compatibility
scikit-learn	1.4.2	Random Forest, HistGradientBoosting, RobustScaler, isotonic calibration, stratified cross-validation, permutation importance, balanced sample weights
XGBoost	2.x	Second ensemble member, gradient-boosted trees
scipy	1.13+	One-sample t-tests for seasonal significance
statsmodels	0.14	Augmented Dickey-Fuller cointegration test for the pairs strategy
transformers	4.40+	FinBERT tokeniser, model and sentiment pipeline
torch	2.2+	FinBERT inference backend
yfinance	0.2.40+	The single market-data source
feedparser	6.0.x	RSS ingestion for news
trafilatura	1.8+	Full-article body extraction, with lxml_html_clean required on modern lxml
google-generativeai	0.7+	Gemini client for the two language-model layers
astral	3.2	Lunar phase indicator in the technical module

Models and external services

Component	Model or endpoint	Purpose and configuration
Primary classifier	`CalibratedClassifierCV` wrapping a 300-tree Random Forest, isotonic, 5-fold	Headline signal. Depth 12, minimum leaf 3, square-root feature sampling, balanced class weights. Isotonic calibration matters because the probability itself is consumed downstream by the portfolio optimiser
Second classifier	`XGBClassifier`, 400 estimators	Depth 5, learning rate 0.05, subsample 0.8, column sample 0.7, L1 0.1, L2 1.5, gamma 0.1, log-loss metric, balanced per-sample weights (XGBoost has no class_weight parameter)
Third classifier	`HistGradientBoostingClassifier`, rule-aware	400 iterations, learning rate 0.05, depth 6, L2 1.0, early stopping on a 10 per cent validation split. Chosen over LightGBM because a Windows LightGBM install was judged too fragile
Sentiment model	`ProsusAI/finbert`	Finance-tuned BERT, three-class. Lazily loaded singleton, roughly 400 MB, 512-token limit handled by sentence-aware chunking and length-weighted probability averaging
Language model	Gemini, `gemini-flash-latest`	Two separate uses: the per-stock insight (temperature 0.4, 2048 output tokens) and the War Room's structured JSON report. The identifier is environment-overridable
Market data	Yahoo Finance via yfinance	Daily OHLCV, company info, financial statements, news. Index symbols: `^NSEI` Nifty 50, `^INDIAVIX`, `INR=X` USD/INR, plus ten NSE sector indices
News sources	Google News RSS and Yahoo Finance RSS	No API key required. Google News links are redirect stubs, so a decoder resolves them through Google's internal batch endpoint before scraping
Charting	TradingView embedded widget	Requires exchange-prefixed symbols, so `INFY.NS` is mapped to `NSE:INFY`

TWO WINDOWS-SPECIFIC ENGINEERING NOTES WORTH RECORDING

Library load order. On Windows, importing torch *after* numpy, scikit-learn or XGBoost triggers a DLL initialisation conflict that silently disables FinBERT. A guarded `import torch` sits at the very top of the entry point purely for its load-order side effect.

Console encoding. The Windows console defaults to cp1252, so an emoji in any `print()` raises mid-request and cascades into a 500 through emoji-printing exception handlers. This was a real production bug on tickers whose sector had no mapped index. The entry point now reconfigures stdout and stderr to UTF-8 with replacement before anything else runs.

SECTION 04

System architecture

How a single request moves through the system

The backend is a thin Flask routing layer over a set of independent analysis modules. The route validates input and normalises the ticker; an orchestrator runs every analyser, isolates their failures, computes the composite score, runs the machine-learning stack, and then layers the decision-support and narrative outputs on top. Nothing in the pipeline can bring down the whole response: each analyser is wrapped so a failure yields a neutral score of 50 and an error marker rather than an exception.

Request lifecycle: POST /api/analyze-stock

#	Stage	What happens
1	Normalise	The ticker is uppercased, any foreign exchange suffix stripped, and `.NS` appended unless `.BO` was given explicitly. This is the mechanism that makes the tool India-only: `AAPL` becomes `AAPL.NS` and fails validation
2	Validate	A five-day history probe plus a quote-type check catches delisted and invalid symbols. The fetcher caches history so validation costs no extra network call
3	Fetch	One fetcher instance retrieves daily history, company info, financial statements and news, caching the history for every downstream consumer
4	Six analysers	Technical, fundamental, sentiment, volumetric, sector and geopolitical run in sequence, each isolated. Sentiment runs with body scraping disabled on this path to keep latency acceptable
5	Composite score	Sub-scores are clamped to 0-100 and weighted. Weights redistribute when the sector analyser fails, and the geopolitical factor claims a 12 per cent slice only when the stock is genuinely exposed
6	Feature vector	The shared feature builder computes 26 features on the latest bar. Real sentiment is merged into the feature dictionary for the rule engine only, never into the model input
7	Tri-model prediction	Three classifiers vote. Probability vectors are mapped to a common class order, soft-averaged into an ensemble, and reported as an explicit consensus with per-model confidences
8	Portfolio	The ensemble's probability of beating the index, the stock's quality score and the investor's appetite and horizon produce a four-bucket allocation
9	Merits and peers	Deterministic pros and cons are derived from the factor picture; the sector cohort is batch-fetched and ranked to surface better performers
10	Narrative	The complete structured picture is compacted into a grounded prompt and Gemini writes a five-to-six sentence Indian-market synthesis
11	Serialise	A recursive sanitiser converts every numpy scalar, pandas object and not-a-number value into JSON-safe primitives before the response is returned

Module topology

Package	Contents	Responsibility
`backend/`	app.py	Eight HTTP routes, ticker normalisation, the JSON sanitiser, the questionnaire risk score, and a six-hour seasonal cache
`backend/modules/`	stock_analyzer, technical_analysis, fundamental_analysis, volumetric_analysis, sector_comparison, geopolitical_analysis, merits_analysis, peer_analysis, seasonal_analysis, trading_strategies, data_fetcher	The orchestrator and every deterministic analyser
`backend/modules/sentiment/`	news_ingestor, finbert_scorer, aggregator, analyzer	The three-phase sentiment pipeline behind one entry point with a thirty-minute per-ticker cache and graceful fallback
`backend/models/`	feature_engineer, data_builder, macro_features, stock_model, portfolio_optimizer, backtest, seasonal_backtest	Feature definition, label construction, the model bundle, allocation logic and the validation harnesses
`backend/services/`	gemini_service, war_service	The two language-model layers and the geopolitical retrieval engine

Package	Contents	Responsibility
`backend/data/geopolitics/`	conflicts, india_relations, trade_exposure, sector_map (JSON)	The hand-curated knowledge base: 13 conflicts, 18 country relations, 17 commodities, 17 transmission themes, 118 mapped tickers
`frontend/src/`	App.jsx, SeasonalAnalysis.jsx, App.css	Three views and around twenty presentational components
`frontend/src/services/`	stockService, warService, seasonalService, healthService, useApi, AppContext	A thin typed service layer plus a generic async hook, so components never manage loading or error state by hand

A DESIGN PRINCIPLE THAT PAID OFF REPEATEDLY

One feature-building function is the only source of truth for the model's inputs, and both training and inference call it. Before this unification, fundamentals were constant placeholders during training but real values at inference, and two technical features were computed with different formulas on each side. The model was effectively being served a distribution it had never seen. Everything downstream - feature importances, calibration, the confidence numbers - was quietly wrong. Collapsing to one function made the mismatch structurally impossible, and the stored model file now carries its feature list so a stale artefact triggers a retrain rather than serving bad predictions.

SECTION 05

The six-factor risk engine

How independent signals become one defensible number

The central output is a 0-100 score where higher means better positioned and lower risk. It is a weighted blend of six independent analysers, each of which produces its own 0-100 score, a categorical signal, and a details payload that the interface renders as an expandable "why this score?" panel. The score is never presented as a black box: the breakdown, the weights actually used, and the reasoning behind each component are all in the response.

Weighting

Factor	Base weight	When exposed to conflict	What it measures
Fundamental	30%	26.4%	Financial health: profitability, efficiency, liquidity, leverage, valuation, governance
Technical	25%	22.0%	Trend, momentum and mean-reversion posture across many indicator families
Sentiment	15%	13.2%	Transformer-scored news mood, stock against its own sector
Volumetric	15%	13.2%	Whether volume confirms the price move; accumulation versus distribution
Sector	15%	13.2%	Relative performance against the correct Indian sector index and the Nifty
Geopolitical	0%	12.0%	Net benefit or harm from tracked conflicts

Two pieces of reweighting logic deserve explanation, because both fix a specific dishonesty:

- **Conditional geopolitical weight.** If the factor always contributed, every unexposed stock would be anchored at a neutral 50 for twelve per cent of its score - a fabricated signal. Instead the factor claims its slice only when the reverse lookup finds real exposure, and the other five weights scale down proportionally.
- **Failure redistribution.** When an analyser fails, returning a neutral 50 and letting it keep its weight silently drags the composite toward the midpoint. The engine detects the failure marker, drops that weight to zero and redistributes it proportionally across the survivors, so a failed component abstains rather than voting neutral.

From score to language

The score maps onto six risk bands, and the same thresholds drive a prioritised recommendations list with explicit flags for weak technicals, weak fundamentals, negative news flow, and both unusually high and unusually low volume. The high-volume flag is worded as a volatility warning rather than a bullish signal, because elevated volume alone does not indicate direction.

Score	75+	65-74	55-64	40-54	25-39	Below 25
Reading	Low risk, strong buy	Low-medium, buy	Medium, hold	Medium-high, caution	High risk, avoid	Very high, sell

Figure 3. Risk bands. The wording is deliberately two-sided: a high score is both low risk and a positive rating.

SECTION 06

Feature deep dives: the analysis factors

For each factor: the problem, the reasoning, the implementation

6.1 Technical analysis

827 lines, ~25 indicator families

The problem it addresses. Retail technical analysis usually means looking at one or two indicators and over-reading them. Any single indicator is regime-dependent: RSI is informative in a range and misleading in a trend, moving-average crossovers work in trends and whipsaw in ranges.

The logic. Compute a wide, diverse indicator set and score the confluence rather than any one reading. Breadth across indicator *families* - trend, momentum, volatility, structure, pattern - means the score reflects agreement across different views of the same price series, which is far harder to fool than a single crossover.

How it is built. A single analyser class computes moving averages, RSI with a zero-division guard, MACD, Bollinger bands, support and resistance, stochastics, VWAP and Heiken-Ashi candles on construction. On top sit detectors for Fibonacci retracements, breakouts from consolidation, reversal patterns, fair-value gaps, candlestick patterns, harmonic patterns, trend lines, Gann angles, Parabolic SAR, Supertrend, RSI divergence, volume profile with a point-of-control, and supply/demand zones. There is even a lunar-phase indicator, included as a curiosity rather than a signal. Several routines were deliberately rewritten from loops into vectorised form - the Heiken-Ashi open and the breakout detector were quadratic before. Every sub-detector in the comprehensive view is wrapped so one failure cannot take down the rest.

What it returns. A 0-100 score, a categorical signal, a trend classification, a details payload for the interface, and a separate machine-learning-safe feature extractor.

6.2 Fundamental analysis

739 lines, five ratio blocks plus governance

The problem it addresses. Price-based analysis says nothing about whether the underlying business is sound. For an Indian retail investor holding through systematic monthly investments over years, business quality matters more than this week's chart. Equally, Indian small and mid-cap investing carries governance risks - promoter behaviour, related-party transactions, pledging - that no ratio captures.

The logic. Score the business across five quantitative dimensions, then add an explicitly qualitative layer for the things that only show up in disclosures and news. Fundamentals carry the single largest weight, 30 per cent, because they are the most persistent of the six signals.

How it is built. Ratios are computed defensively: a helper tries multiple possible line-item keys for each statement value, because Yahoo Finance labels vary by company, and any statement may be missing entirely. The five blocks are profitability (margins, return on equity), efficiency (turnover), liquidity (current and quick ratios), leverage (debt-to-equity, interest coverage) and valuation (price-to-earnings, price-to-book), each scored against benchmarks and contributing a capped share of the total. The qualitative layer scans recent news text for keyword signatures covering management background, business ethics, corporate governance, minority-shareholder treatment, insider

share transactions, related-party transactions, promoter compensation, operator activity, shareholding patterns and political affiliation, and a dedicated red-flag detector aggregates the serious ones.

What it returns. A 0-100 score with a per-category breakdown that the interface shows as "Profitability: 25/25"-style lines, plus a signal and the full comprehensive analysis.

6.3 Enhanced sentiment

Three-phase pipeline, 936 lines, FinBERT

The problem it addresses. The original implementation counted positive and negative keywords in headlines. That fails on exactly the sentences that matter: financial language is full of negations, hedges and constructions where the sentiment-bearing word is ambiguous out of context. Worse, an isolated stock sentiment score is uninterpretable - if a pharmaceutical stock reads negative, the useful question is whether the whole pharmaceutical sector reads negative, or just this company.

The logic. Three separable phases. Source news for both the stock and its sector; score with a model actually trained on financial text; aggregate with weights that reflect how news sentiment behaves - it decays fast and off-topic articles should count less. Then report the *divergence* between the stock's mood and its sector's, which is the signal a human analyst would actually look for.

How it is built. Phase 1, ingestion. Google News RSS queries built from the company's real resolved name plus Yahoo's per-ticker feed for the stock scope, and an industry-level query for the sector scope, restricted to a seven-day window. A relevancy filter tokenises the company name with corporate suffixes stripped, so articles about an unrelated entity sharing the name score low and are dropped below a threshold. Articles are deduplicated on a normalised title, ranked by relevance then recency, trimmed per scope, and tagged with their scope. Google News links are not redirects but opaque stubs, so a decoder extracts the article signature and timestamp from the page and resolves the real publisher URL through Google's internal batch endpoint before trafilatura extracts the body. **Phase 2, scoring.** FinBERT is loaded once as a lazy singleton. Text beyond the 512-token limit is split sentence-aware into word chunks, each scored independently, and the class probabilities combined as a length-weighted average - so a long article is not judged on its first paragraph. The mapping onto the app's scale is 50 plus the positive-minus-negative probability gap times 50, which puts genuinely neutral coverage at exactly 50. **Phase 3, aggregation.** Each article's weight multiplies a recency decay with a three-day half-life by a relevance term. Directional agreement across articles is computed as one minus normalised dispersion, and the reported confidence blends sample size, agreement and model confidence. The stock-minus-sector gap becomes a labelled divergence signal with a plain-language note.

What it returns. The legacy score, sentiment label and confidence keys are preserved exactly, so the whole pipeline was a drop-in replacement; added on top are the sector sentiment block, the divergence signal, and a reasoning object listing the specific articles that most drove each side. If FinBERT or the news feeds are unavailable, the layer falls back to the keyword engine and then to a neutral result, so the output contract always holds.

> **WHY THE DIVERGENCE SIGNAL IS THE INTERESTING PART**
>
> A stock sentiment of 42 out of 100 means very little on its own. A stock at 42 while its sector sits at 61 means something specific: the market has a problem with this company, not with its industry. The reverse - a stock at 61 in a sector at 42 - is company-specific optimism worth understanding. This framing came directly from the observation that sector-level factors drive individual automobile stocks, and it is the signal that a raw per-stock score cannot express.

6.4 Volumetric analysis

217 lines, fully vectorised

The problem it addresses. Price moves without volume behind them are unreliable. A breakout on thin volume is frequently a failed breakout, and a decline on heavy volume means something different from a drift down on no participation.

The logic. Score whether volume *confirms* the price action, and whether the accumulation-distribution balance suggests buyers or sellers are in control. Four sub-signals contribute a capped number of points each, summing to 100.

How it is built. On-balance volume, a price-volume trend series and an accumulation-distribution line are all computed on construction, each fully vectorised - on-balance volume as the cumulative sum of signed volume rather than a Python loop with positional writes. Every division carries an explicit guard: zero previous closes are replaced before percentage change, and doji candles where high equals low are handled in the close-location value. The four scored sub-signals are the volume trend against its recent average (up to 35 points), the on-balance volume direction read as accumulation or distribution (25), price-volume confirmation (25), and bullish volume patterns such as up-days on rising volume and volume spikes (15). A guard checks up front that a usable volume column exists at all and returns a neutral score with an error marker if not, rather than producing nonsense from zeros.

What it returns. A 0-100 score, a five-level signal from very bearish to very bullish, and the four sub-signal breakdowns with their statuses.

6.5 Sector comparison

236 lines

The problem it addresses. "This stock returned twelve per cent over three months" is not information until you know what its sector and the index did. Comparing an Indian stock against the S&P 500, which a naive implementation would do by default, is actively misleading.

The logic. Resolve the correct benchmark from the ticker's own exchange suffix, then measure the stock against both its sector index and its market index. Relative strength against the right benchmark is one of the few genuinely robust equity signals.

How it is built. The exchange suffix drives a market-index lookup - NSE maps to the Nifty 50, BSE to the Sensex, with sensible mappings for other exchanges and the S&P 500 only as a last-resort default. A separate Indian sector map points at the real NSE sector indices for technology, financials, healthcare, energy, consumer cyclical and materials, alongside a US SPDR map retained for completeness. Period returns are computed so that a data failure returns nothing rather than zero, letting callers distinguish "flat" from "unknown" - a distinction the composite score depends on for its reweighting logic. A rival-comparison path can rank named peers directly.

What it returns. A 0-100 relative score, the resolved sector, and versus-market and versus-sector outperformance figures that also feed the merits panel.

6.6 Geopolitical exposure

87 lines over a 31 KB knowledge base

The problem it addresses. Indian equities are unusually exposed to geopolitics: roughly 85 per cent of crude is imported, a large share of pharmaceutical ingredients comes from China, and more than half of edible oil is imported. A Strait of Hormuz incident is a direct earnings event for Indian aviation, paints and tyres. Yet this exposure appears in no standard screening metric, and most tools treat geopolitics as commentary rather than as a scoreable factor.

The logic. Invert the War Room's knowledge base. Instead of asking "which stocks does this conflict affect?", ask of a given ticker "which tracked conflicts touch it, and in which direction?" Weight each exposure by how live the conflict is and how relevant it is to India, then net the benefits against the harms. Crucially, this is deterministic - no language-model call - so it adds negligible latency to every analysis.

How it is built. A reverse index is built from the sector map: every ticker mentioned in any theme's benefits or harms lists maps to its themes, directions, sectors and rationales. For a given ticker, each tracked conflict's activated theme set is intersected with that ticker's themes to produce concrete exposures. Each exposure's weight

is the product of a status multiplier - active 1.0, recurring 0.9, frozen 0.6, latent 0.4 - and an India-relevance multiplier from 1.0 down to 0.4. The signed weighted mean gives a bias in the range minus one to plus one, mapped to a score of 50 plus bias times 40, which lands in roughly 10 to 90. Drivers are deduplicated per conflict, direction and sector for display.

What it returns. A 0-100 score, a signal of geopolitically favoured, exposed or mixed, an exposure flag that gates the composite weighting, a conflict count, and up to eight named drivers each linking a conflict to a sector with a written rationale.

SECTION 07

The machine-learning core

Target design, feature engineering, three models, and how they vote

7.1 The target: the most important decision in the project

The original model predicted absolute direction as three classes - a forward return above three per cent was a buy, below minus three per cent a sell, anything else a hold. It reached about 0.60 cross-validated accuracy against a 0.59 majority baseline. That is not a model. It is an expensive way to predict the majority class.

The diagnosis is structural rather than a matter of tuning. Absolute stock returns are dominated by market-wide drift. A model predicting them spends its capacity re-predicting the index, which every stock shares, and the stock-specific component - the only part with any information for stock selection - is drowned out. The class distribution compounded it: a hold-heavy three-class problem structurally caps confidence near the majority rate.

THE CURRENT TARGET

```
excess = stock 20-day forward return − Nifty 50 20-day forward return
label = 1 (OUTPERFORM) if excess > 0, else 0 (UNDERPERFORM)
```

This answers the question an investor picking Indian equities actually asks - "will this beat the index?" - rather than "will it go up?", which is mostly a question about the market. Subtracting the benchmark strips out market drift and forces the model to learn stock-specific edge. Two near-balanced classes give a clean baseline near fifty per cent and confidence numbers that mean something.

Target design	Samples	Baseline	Best CV accuracy	Edge over baseline	ROC-AUC
Absolute 3-class, 5-day horizon	176,893	0.590	0.601 (XGBoost)	+1.1 pts	not reported
Absolute 3-class, 20-day horizon	175,248	0.490	0.563 (Hybrid)	+7.3 pts	not reported
Binary relative, 20-day horizon	**173,000**	**0.516**	**0.626 (Random Forest)**	**+11.0 pts**	**0.811**

Figure 4. Target evolution. The middle row looks worse than the first on raw accuracy while actually being better - it clears its baseline by seven points where the first clears its own by one. Comparing accuracies across different baselines is exactly the trap this table is designed to expose.

THE CAVEAT THAT MUST TRAVEL WITH EVERY PREDICTION

OUTPERFORM does not mean "goes up". In a falling market it can mean losing money, just less than the index would have lost. The interface, the merits panel and the language-model prompt all state this explicitly, because a relative signal read as an absolute one is worse than no signal at all.

7.2 Features: 26 columns, every one reconstructable

The governing constraint is that every feature must be computable from OHLCV history for any historical bar. This is what guarantees that the distribution the model trains on is the distribution it is served. Fundamentals and sentiment

are deliberately excluded, not from lack of ambition but because free data exposes only about five quarters of fundamentals - they cannot be reconstructed across five years of daily bars, and including them would reintroduce the train/serve mismatch. They remain first-class factors in the composite score and sentiment still feeds the rule engine; they are simply not model inputs.

Group	Features	What they capture
Core momentum and trend	returns, momentum_5d, momentum_20d, momentum_60d, roc_10, price_vs_sma50, market_trend, trend_strength	Direction and persistence across horizons from days to a quarter
Oscillators	rsi, stoch_k, cci, macd_hist, bollinger_position	Overbought and oversold positioning; the Bollinger term is a clipped z-score rather than a raw band position
Volatility and risk	volatility, atr_pct, vol_regime, downside_dev, return_skew, sharpe_ratio	Level and character of risk. The volatility regime is the ratio of ten-day to sixty-day realised volatility; downside deviation isolates negative returns; skew captures return asymmetry
Volume	volume_change, volume_ratio, obv_trend	Participation, normalised by recent turnover so the on-balance-volume term is scale-free across stocks
Structure	dist_52w_high, dist_52w_low	Position within the annual range - where a stock sits relative to its own extremes
Market context	vix_z, rs_20d	The India VIX z-score as a fear-regime indicator, and relative strength as the stock's twenty-day momentum minus the Nifty's

A NEGATIVE RESULT WORTH MORE THAN A POSITIVE ONE

The first attempt at market context added the absolute series too: Nifty twenty-day return, Nifty trend, Nifty volatility, VIX level and the rupee move. They immediately became the top-ranked drivers and homogenised every prediction toward the market's direction - every stock returned the same signal and stock-to-stock differentiation collapsed. The cause is obvious in hindsight: those columns are *identical for every stock on a given date*, so a tree model finds them irresistible and learns to predict the market instead of the stock. They were removed. Only the stock-specific relative strength and the regime-level VIX z-score survived. The Nifty return is still computed internally, purely to derive relative strength.

7.3 Three models, and why each one is there

The ensemble is not three copies of the same idea. Each member is included for a distinct reason, and their disagreement is treated as information rather than hidden behind an average.

Model	Why this model	Configuration and notes
Calibrated Random Forest	Bagged trees are robust to the noise that dominates financial features, and the isotonic calibration wrapper matters because the probability itself is consumed downstream - the portfolio optimiser sizes positions from it, so a miscalibrated 0.7 would produce a wrong allocation, not just a wrong label	300 trees, depth 12, minimum split 6, minimum leaf 3, square-root feature sampling, balanced class weights, isotonic calibration over 5 folds. Feature importances are recovered by averaging across the calibrated sub-estimators, since the wrapper hides them
XGBoost	Boosting finds interactions that bagging misses, giving the ensemble a genuinely different inductive bias rather than a second opinion from the same family	400 estimators, depth 5, learning rate 0.05, subsample 0.8, column sample 0.7, L1 0.1, L2 1.5, minimum child weight 3, gamma 0.1, log-loss metric. Class balance is handled with per-sample weights because XGBoost has no class-weight parameter

Model	Why this model	Configuration and notes
Rule-aware hybrid	The project also contains a hand-built technical rule engine. Rather than letting rules and models argue in the interface, this member trains on the features *plus the rule engine's own net score* as a 27th column, so it learns where the rules are trustworthy and where they should be discounted	HistGradientBoosting, 400 iterations, learning rate 0.05, depth 6, L2 1.0, early stopping on a 10 per cent split. The rule column is computed by the same vectorised function at training time and the same scalar function at inference, from unscaled feature values, so it stays consistent. Native importances are unavailable, so permutation importance is computed once at training on a 4,000-row subsample and persisted

7.4 The rule engine and the shift from override to nudge

A twelve-condition scoring function reads RSI extremes, returns, the Sharpe ratio, market trend, the Bollinger z-score, sentiment and a volatility-plus-negative-returns combination. It returns a directional signal only when at least two conditions align, and otherwise abstains and lets the models decide.

Its role changed deliberately. It used to hard-flip the model's label when it disagreed, which meant a confident, well-calibrated model could be overruled by a handful of thresholds. Now it nudges the opposing probability by 0.25 and recomputes the argmax. A genuinely confident model survives the nudge; a borderline one gets tipped. The raw model vote is preserved separately for the consensus display, so the interface can show both what the model said and what the rules said.

7.5 Soft voting and honest consensus

Each model's probability output is first mapped into a common class order - necessary because scikit-learn and XGBoost do not guarantee the same internal ordering, and getting this wrong would silently invert predictions. The vectors are then averaged. Averaging cuts each model's variance, so when the members broadly agree the combined confidence is both higher and better calibrated than any single member's.

The response reports the ensemble prediction, its confidence, and the single most interpretable number in the system - the probability of beating the Nifty. Alongside it sits an explicit consensus object: each model's name, raw vote and confidence, whether all three agree, the agreement ratio, the majority vote, and the rule engine's separate signal. When the models split, the interface says "split, two of three, majority outperform" rather than presenting a smoothed average as though it were unanimous.

7.6 Training and persistence

- **Universe.** 153 NSE tickers in a Nifty-200-style spread across seventeen sector groups; three fail Yahoo Finance on symbol quirks, so 150 train.
- **Data.** Five years of daily bars per ticker, real only. If the fetch produces nothing, training raises rather than falling back - the synthetic generator was deleted by design.
- **Labels.** The Nifty forward return is computed on the index's own trading calendar and then aligned onto each stock's dates with forward fill, so holiday mismatches cannot corrupt the excess return.
- **Scaling.** RobustScaler, chosen over standard scaling because financial features have heavy tails and outliers that would distort a mean-and-variance fit.
- **Evaluation.** Five-fold stratified cross-validation per model, plus ROC-AUC as the headline metric - it measures how well the model *ranks* outperformers independently of where the 0.5 cutoff falls.
- **Persistence.** The saved bundle is a versioned dictionary carrying the feature list. Loading rejects a mismatched version or feature set and retrains, so a stale artefact can never be served against a changed feature definition. Training takes roughly five to eight minutes and requires network access.

SECTION 08

Decision-support layers

Turning a score and a probability into something a person can act on

8.1 Portfolio optimiser

222 lines, profile-driven

The problem it addresses. A score and a probability do not answer the actual question, which is "how much of my money, if any?" And the honest answer differs for a 25-year-old with a twenty-year horizon and an aggressive appetite versus a 58-year-old prioritising capital preservation, even for the identical stock.

The logic. Size the equity sleeve from the model's *edge* over the index rather than its raw probability, scale it by the stock's own quality, then cap and shape it with the investor's stated appetite and horizon. The output is a four-bucket allocation in instruments an Indian retail investor actually uses.

How it is built. The base weight starts at 0.30 for a coin-flip read and scales with the doubled edge, reaching roughly 0.65 at maximum conviction - deliberately scaling the edge rather than the probability, so a 50/50 read earns a modest position and only genuine conviction earns a heavy one. A quality factor of 0.4 plus 0.6 times the normalised risk score multiplies it, so a healthier stock earns more weight. This corrected an inverted bug: the earlier formula used one minus the score, which starved the best stocks of allocation. Appetite then imposes a hard equity ceiling of 35, 55 or 80 per cent, nudged up ten points for horizons of fifteen years or more, five points beyond seven years, and down ten points at two years or less. The defensive remainder splits across debt and government securities, gold through sovereign bonds or exchange-traded funds, and liquid cash, in proportions that also vary by appetite.

What it returns. A four-line allocation, a Herfindahl-based diversification score computed as the effective number of asset-class positions normalised to 0-100, a plain-language concentration label, contextual suggestions, and a prescriptive guidance block on how the investor's profile *should* be shaped to hold a stock of this risk character.

> **A SMALL FIX THAT MATTERED**
>
> The diversification score was originally 100 minus the distance from a fifty per cent equity weight, which peaked at exactly the half-in-one-stock allocation - rewarding concentration as though it were balance. It is now one over the sum of squared weights, normalised, which is the standard effective-positions measure. A heavy single-name tilt now scores low, as it should.

8.2 Merits and demerits

147 lines, fully deterministic

The problem it addresses. Factor scores are abstract. "Fundamental: 43" does not tell a retail investor what to do with the information, and different investors care about different things - a momentum trader and a systematic-investment-plan holder read the same stock completely differently.

The logic. Translate the computed picture into two plain-language lists framed around what investors actually want: momentum, financial strength, smoothness of the ride, relative strength, the crowd's mood, macro exposure, and the quant read. Every item is derived from a threshold on a computed value - no language model, so it cannot hallucinate a reason.

How it is built. Each factor score passes through a tone classifier with separate good and bad thresholds, and each tone emits a titled item with a written detail that quotes the actual number. Volatility bands, sector outperformance thresholds, the sentiment divergence label, named geopolitical drivers and the model consensus each contribute their own entries. Model disagreement is itself listed as a demerit - a reason to wait for cleaner confirmation. Fallback entries guarantee neither panel is ever empty.

What it returns. Two lists of titled items with details. The framing is deliberately non-directive: it explains who a stock suits and why, never instructing anyone to buy or sell.

8.3 Peer alternatives

162 lines, batch-fetched and cached

The problem it addresses. Telling someone their stock scores 41 out of 100 is unhelpful without the follow-up: if not this, then what? Concretely, if a large IT name is lagging, the investor wants to know which comparable IT names are not.

The logic. Rank the stock's own sector cohort by recent return, establish whether it sits below the cohort median, and if so surface the better performers as immediately actionable alternatives.

How it is built. Fifteen curated NSE sector cohorts mirror the training universe groups, so the peer set is liquid and sensible rather than whatever a sector string returns. Membership resolves the group directly, with a coarse Yahoo Finance sector mapping as fallback. The whole cohort's six-month closes are fetched in a single batch call and cached for an hour, which is what makes the feature fast enough to run on every analysis. The analysed stock's own return is passed in from the already-fetched history to avoid a redundant download. Underperformance is defined against the cohort median, and alternatives are the top peers that beat the analysed stock.

What it returns. The sector group, the stock's own return and rank within the cohort, the cohort median, an underperformance flag, and up to three alternatives rendered as clickable chips that immediately re-run the full analysis on that peer.

8.4 The Gemini insight layer

167 lines, prompt and context engineering

The problem it addresses. Six factor scores, three model votes, a divergence signal, geopolitical drivers and an allocation is a lot of surface area. A reader needs the synthesis: what matters most here, what argues against it, and what Indian macro context bears on it. That synthesis is genuinely hard to write with rules, because the important combination differs per stock.

The logic. Use a language model strictly as a narration layer over evidence that has already been computed. There is no fine-tuning; the India-awareness comes entirely from the role definition and the structured context. The model is instructed to reason only from the supplied data, and every number it can quote is already in that data.

How it is built. A context builder compacts the whole result into labelled lines: the score and risk level, the six-factor breakdown, the three model votes with confidences and the ensemble's probability of beating the index, technical and fundamental signals, the sentiment divergence, sector relative performance, named geopolitical channels with directions, the personalised allocation with its profile basis, and the rule-based rating. The prompt casts the model as a Mumbai sell-side analyst thinking in rupees and in the Indian macro frame - the repo rate cycle and liquidity, foreign versus domestic institutional flows, the rupee, the monsoon and rural demand, the Union Budget and goods-and-services tax, festive and wedding-season demand, and the relevant Nifty sector index. It specifies a five-part structure, demands the relative nature of the model call be stated precisely, requires one clear positive and one clear risk, forbids buy-and-sell directives and hedging filler, and mandates plain prose. Temperature is 0.4 for consistency. The output budget is 2048 tokens because this is a thinking model whose internal reasoning draws from the same budget - a small cap truncates the visible answer mid-sentence.

What it returns. A five-to-six sentence grounded narrative closing on an educational-analysis caveat. A failure returns a message, never a fabricated insight, so the rest of the analysis is unaffected.

SECTION 09

The War Room

Retrieval-grounded geopolitical impact analysis for Indian equities

This is the most distinctive module in the project. It answers a question no standard equity tool addresses: if a specific conflict escalates, which Indian sectors and which named NSE tickers benefit, which suffer, and through what mechanism?

Why retrieval rather than a trained model

Geopolitical impact reasoning has no labelled dataset worth training on - conflicts are few, heterogeneous and non-stationary. Asking a language model unaided produces fluent, plausible and unverifiable output that will happily invent tickers. The chosen approach is retrieval-augmented generation over a hand-curated knowledge base: the facts are auditable and editable, the model supplies only the reasoning, and every claim is traceable to a knowledge-base entry.

The knowledge base

File	Size	Contents
`conflicts.json`	8.9 KB	13 conflicts, each with parties, status, start year, India-relevance, a written summary, key commodities and explicit escalation triggers. Covers Russia-Ukraine, India-Pakistan, Israel-Iran, China-Taiwan, India-China, the US-China technology war, Red Sea and Houthi disruption, Middle East and OPEC dynamics, the South China Sea, Venezuela sanctions, North Korea, Bangladesh instability and the Myanmar conflict
`india_relations.json`	7.7 KB	18 bilateral country profiles describing India's actual relationship with each party - the reasoning substrate for why a conflict transmits to India the way it does
`trade_exposure.json`	4.7 KB	17 commodities with quantitative Indian import and export dependency context - crude at roughly 85 to 88 per cent imported, pharmaceutical ingredients at 60 to 70 per cent from China, edible oil at 55 to 60 per cent imported
`sector_map.json`	10.0 KB	17 transmission themes, each listing the Indian sectors that benefit and those that suffer, with named NSE tickers and a one-line rationale per entry. 140 ticker mappings across 118 unique tickers

The transmission model

The link from conflict to sector is not left to the language model. A commodity-to-theme mapping encodes it explicitly: crude oil activates both a price-spike theme and a discounted-feedstock relief theme; semiconductors activate supply disruption and the China-plus-one substitution theme; rare earths activate a supply squeeze. Two additional rules add channels that commodity lists alone would miss - conflicts where India is itself a belligerent add defence indigenisation and a risk-off rupee-weakness channel, and any active conflict with high India-relevance adds the classic risk-off winners, information-technology exporters and gold.

The analysis pipeline

1. **Resolve the scenario.** Either a conflict identifier from the curated watchlist, or free text. Free text is grounded by matching known country names and commodity terms from the knowledge base, so a user-typed scenario still carries real facts into the prompt rather than being answered from thin air.
2. **Retrieve.** Gather the conflict entry, both parties' India-relations profiles, the trade-exposure entries for every relevant commodity, and the candidate sector and ticker maps for every activated theme.
3. **Assemble.** Serialise those facts as JSON into a prompt that casts the model as a geopolitical risk analyst for Indian equity investors, instructs it to prefer the listed tickers, and requires India-specific reasoning through bilateral relations and trade exposure.

4. **Generate.** The model returns strict JSON to a specified schema: an India summary, a net bias of risk-on, risk-off or mixed, transmission channels each with a driver mechanism and direction, affected sectors with impact polarity, a magnitude from one to five, a rationale and tickers, plus watch items, a confidence level and caveats.
5. **Parse defensively.** An extractor strips markdown fences and falls back to the outermost braces. If parsing still fails, the raw text is returned with a note rather than an error, so a malformed response degrades gracefully.
6. **Report grounding.** The response includes which commodities, country profiles and themes the report was grounded on, so a reader can audit the evidence base.

VERIFIED BEHAVIOUR

Russia-Ukraine correctly produces refiners benefiting from discounted feedstock, oil marketing companies, aviation, paints and tyres hurt by a crude spike, defence names benefiting from indigenisation, and fast-moving consumer goods and fertilisers hurt through edible oil and urea feedstock. The free-text scenario "Iran closes the Strait of Hormuz" produces a coherent risk-off report. Clicking any ticker chip in a report jumps to the analysis view and immediately runs the full per-stock analysis on it - the two halves of the system connect.

SECTION 10

Seasonal analysis

Market-relative, significance-tested sector rotation

The Indian market has genuine calendar structure: the Union Budget in February, financial-year end in March, new-year optimism and fourth-quarter earnings in April, the monsoon from June, the festive season peaking around Diwali in October and November. The question is whether any of it is statistically real.

The flaw in the original implementation

WHAT WENT WRONG, AND WHY IT IS INSTRUCTIVE

Version one averaged each sector's raw monthly returns over ten years and surfaced the best months. It produced a result that looked like a finding: nearly every sector peaked in April. That is not sector seasonality - it is market beta. April is historically the Indian market's strongest month, so every sector rises in April, and averaging raw returns simply rediscovers the market's own calendar in ten different disguises. The output was confidently presented and analytically empty.

The three corrections

1. **Excess returns against the Nifty 50.** Each sector's monthly return minus the benchmark's return for the same month. This strips out market beta and measures true rotation - which sector *beats* the market in a given month, not which one goes up. The April artefact disappears.
2. **Statistical significance.** A one-sample t-test on each sector-month's excess returns against zero. Patterns are surfaced as reliable only when they clear a p-value bar of 0.10, and the ranking uses the t-statistic - a signal-to-noise measure of seasonal edge - rather than the arbitrary composite score the first version used. Significance is displayed with conventional star notation.
3. **The Indian event calendar.** Every pattern is tagged with the real India-specific driver. A month-level calendar covers all twelve months, and a sector-month table adds specific drivers - festive vehicle demand from Navratri to Diwali for automobiles in October, monsoon-driven rural demand for consumer goods in June and July, the year-end government capital-expenditure push for infrastructure in March, monsoon-season acute-therapy demand for pharmaceuticals in July.

Implementation

- Ten sectors, each resolved by trying its NSE sector index first and falling back to an equal-weighted average of component closes - so a failed index fetch degrades rather than dropping the sector.
- Ten years of history, resampled to month-end closes, converted to percentage monthly returns and joined to the benchmark on year and month.

- Per sector-month: mean and standard deviation of both raw and excess returns, the positive rate, the win rate against the market, the t-statistic, the p-value and a significance flag. Quarterly aggregates alongside.
- The best three-month window is found by scanning all twelve rolling windows for the highest summed t-statistic.
- Top picks are restricted to sectors with genuinely positive excess in the current month, ranked by edge, with confidence derived from the p-value and sample count and a written reason quoting the actual figures.
- Two heatmaps - raw and market-relative - with the interface defaulting to the market-relative view, plus a year-round list of every statistically significant edge across all sectors and months.
- Results are cached server-side for six hours, since the underlying ten-year statistics do not move intraday and the computation involves many fetches.

What it actually found

Eleven statistically significant edges across the ten sectors, including both positive and negative ones - the negatives matter, because a method that only ever finds good news is not testing anything:

Sector and month	Mean excess vs Nifty	p-value	Years beating market	Indian driver
Automobiles, April	+2.73%	0.022	81.8%	New financial year and fourth-quarter earnings
Information technology, December	+2.24%	<0.05	high	Year-end drift on thin volumes; rupee seasonality
Banking, October	+2.17%	<0.05	high	Festive credit demand around Diwali
Information technology, April	–3.76%	<0.05	low	Rotation away from exporters into domestic cyclicals

Figure 5. A selection of the eleven significant edges found. For a month with no significant edge the module explicitly reports low confidence and no seasonal tailwind rather than recommending noise.

SECTION 11

Trading strategies and backtester

A quantitative strategy engine, currently reachable only through the API

An earlier phase built a complete strategy layer with four strategy families, a backtester and a risk manager. It was removed from the interface when the product focus narrowed to analysis rather than execution, but the code and its two endpoints remain and work. It is documented here both because it is real engineering and because it is the most immediately revivable asset in the repository.

Strategy	Method	Position sizing
Trend following	Moving-average structure filtered by the Average Directional Index, so signals only fire when a trend is strong enough to be worth following. Includes a cross-sectional momentum ranking across a universe	Volatility-scaled
Mean reversion (pairs)	Spread z-score between two instruments, with an augmented Dickey-Fuller cointegration test to check the pair is statistically suitable before trading it - the step most naive pairs implementations skip	Hedge-ratio matched across both legs
Technical indicator	Confluence of RSI, MACD and Bollinger bands with Average True Range for stops	Risk-per-trade based on the ATR stop distance
Calendar spread	Spread dynamics between near-month and next-month contracts	Spread-notional based

Backtester and risk manager

The backtester runs a strategy over historical data with a configurable initial capital and reports final capital, total return, trade count, wins and losses, win rate, average win and loss, maximum drawdown and the Sharpe ratio, plus

a capped trade log. The risk manager provides Average-True-Range and percentage-based stop losses, trailing stops and risk-reward computation. A selector factory instantiates strategies by name so the API can accept a strategy type and parameter dictionary generically.

Two endpoints remain live

- `POST /api/backtest-strategy` - validates the ticker, strategy type and a positive initial capital, fetches two years of data, requires a pair ticker for the strategies that need one, runs the backtest and returns the full metric set with a sanitised trade list.
- `POST /api/strategy-signals` - fetches one year of data, generates signals, and returns the latest signal as buy, sell or hold with the current price and a computed position size.

SECTION 12

Validation and honest measurement

Two harnesses built specifically to try to disprove the system's claims

Cross-validated accuracy on the training panel is a weak claim for a financial model, because financial data is temporally ordered and shuffled folds leak information across time. Two dedicated harnesses exist to test the system the way a sceptic would.

12.1 Out-of-sample model backtest

- **Time split, not ticker split.** A fresh model trains only on bars dated on or before a cutoff and is evaluated on bars strictly after it. Using the same liquid universe on both sides is deliberate and honest, because the test *period* is unseen - which is the leakage that actually matters.
- **Label-safe buffer.** Because the label looks forward, the final bars before the cutoff are dropped so no training label can peek past it. This is the single most commonly botched detail in retail backtesting.
- **Identical features.** The harness calls the same feature builder production uses, so it validates the deployed pipeline rather than a reimplementation.
- **Economic edge, not accuracy, as the headline.** The reported metrics are the mean forward return of buy-tagged bars versus sell-tagged bars versus the all-bar baseline, the hit rate against the base rate, and the buy-minus-sell spread. A model can be accurate and worthless; the spread is what would show up in a return stream.

12.2 Seasonal edge persistence test

This harness exists to answer the obvious objection to any seasonality claim: that it is curve-fitting. Each sector's history is split into an older in-sample portion and a recent out-of-sample portion. Significant edges are discovered on the in-sample data only, using exactly the procedure the live tool uses. Then the out-of-sample data is checked: did an outperform edge keep a positive mean excess, and an underperform edge a negative one? The aggregate persistence rate and the out-of-sample excess of flagged edges are compared against an all-cells baseline.

The harness also states its own limitation prominently: seasonality gives only about ten annual observations per month, so per-edge out-of-sample samples are tiny - around four. The aggregate across all flagged edges is the meaningful read; individual edges are directional, not definitive. Building a validation tool that reports its own statistical weakness is the point.

12.3 Current measured results

Measurement	Result	Interpretation
Random Forest cross-validated accuracy	**0.626**	Against a 0.516 majority baseline - an 11-point edge, the first genuine one across all target designs tried
Ensemble ROC-AUC	**0.811**	The model ranks outperformers well independently of the 0.5 cutoff. This is the honest headline for a binary target
XGBoost cross-validated accuracy	0.602	Consistent with the forest; different bias, similar skill

Measurement	Result	Interpretation
Hybrid cross-validated accuracy	0.606	The rule-aware member is competitive, so the hand-built rules carry real information rather than noise
Training panel	173,000 samples	150 NSE tickers, five years of daily bars, classes at 83,600 and 89,300 - near balanced by construction
Significant seasonal edges	11	Across ten sectors and twelve months at p below 0.10, including negative edges
Live prediction spread	34.9% to 73.9%	Verified probabilities of beating the index across three tickers, producing equity weights from 15.3 to 35.5 per cent - the system genuinely differentiates rather than returning one answer

SECTION 13

Frontend and user experience

One page, three views, and an interface that shows its own uncertainty

Structure

- **App shell.** Three views - Analysis, War Room and Seasonal - switched by state rather than a router, with a backend health indicator rendered as a coloured dot and an explicit offline banner.
- **Control panel.** Ticker input, a segmented risk-appetite control and a horizon slider. The appetite and horizon are not decoration: they are sent to the API and drive the allocation.
- **Result composition.** An overall-score hero, an embedded TradingView chart, the six-factor grid, the three-model artificial-intelligence section, the consensus panel, the portfolio card, the merits panel, peer alternatives and the recommendations list.
- **Expandable factor cards.** Each factor card carries a "why this score?" disclosure built from the backend details payload, with tone-coloured dots driven by a keyword classifier over status strings.
- **Cross-view handoff.** Clicking a ticker chip in a War Room report sets a handoff object with a timestamp, switches to the analysis view and auto-runs the analysis. The timestamp is what makes re-clicking the same ticker work.
- **Skeleton loading.** A cold analysis takes around 25 seconds, so the result area renders a skeleton rather than a spinner.

Service layer

Components never call HTTP directly. Six thin service modules wrap the endpoints and a generic async hook exposes data, loading, error and execute for any of them, so no component manages request state by hand. A context provider holds shared state including backend health and the last analysis result.

A BUG WORTH RECORDING, BECAUSE IT WAS INVISIBLE

The async hook guards against setting state after unmount using a mount reference. That reference was originally initialised but never driven by the mount lifecycle - its cleanup function was returned to callers but never wired to an effect. Under React 18 strict mode the initial mount is an unmount followed by a remount, so the reference could get stuck false and *silently drop a successful response*. The request returned 200, the data arrived, and nothing rendered. This caused a long stretch of unexplained blank results during testing. The fix is a three-line effect that sets the reference true on mount and false on unmount.

Design system

A restrained fintech aesthetic: muted navy-slate surfaces, a single blue accent, Inter for text with IBM Plex Mono for figures, subtle borders and shadows, and no neon or animation. Light and dark modes are supported through both the system colour-scheme preference and a data attribute. The overall-score card was deliberately changed from a bright colour wash to a neutral background with a coloured accent bar and a bold coloured number, because the wash was making the most important figure on the page harder to read.

Presenting uncertainty as a first-class element

The interface is designed to show doubt rather than hide it. The consensus panel displays each model's individual vote and confidence, states plainly when the models are split and which side holds the majority, shows the rule engine's cross-check as a separate line, and renders three separate driver-importance charts - one per model - rather than a single blended ranking that would imply more agreement than exists. Model disagreement additionally appears as an explicit demerit in the merits panel.

SECTION 14

Novelty: what is genuinely different

Seven design decisions that are defensible, and one honest accounting of what is not novel

Plenty of tools compute RSI. What follows is the set of decisions that would be hard to find combined in one system, each stated with the reasoning that makes it more than a feature list entry.

#	The decision	Why it is genuinely different
1	**A relative-performance target instead of absolute direction**	Almost every retail-facing model predicts up or down, which is dominated by market drift and therefore mostly re-predicts the index. Framing the target as excess return against the Nifty subtracts market beta at the label level, not as a post-hoc adjustment, and forces the model to learn stock-specific edge. It also matches the real decision - this stock versus simply buying the index. The measured jump from a one-point to an eleven-point edge over baseline is the evidence
2	**A rule-aware ensemble member**	Systems normally either use rules or use models, and when both exist they argue in the interface. Feeding the rule engine's own net score to a third model as an extra feature lets the system *learn where its own rules are trustworthy*. That member is competitive with the pure models, which is itself a finding: the hand-built rules carry real information
3	**Geopolitics as a scoreable, deterministic per-stock factor**	Geopolitical risk is normally commentary. Here a curated knowledge base of 13 conflicts, 17 commodities and 17 transmission themes over 118 mapped tickers is inverted into a reverse lookup, weighted by conflict status and India-relevance, and folded into the composite score with no language-model call. This is unusually relevant for India specifically, given import dependency in crude, pharmaceutical ingredients and edible oil, and it appears in no standard screening metric
4	**Sentiment divergence rather than an isolated sentiment score**	A stock sentiment number alone is uninterpretable. Scoring the stock and its sector separately with a finance-tuned transformer and reporting the gap answers the question a human analyst asks: is this a company problem or an industry problem? The recency half-life and relevance weighting reflect how news sentiment actually behaves
5	**Significance-tested, market-relative seasonality**	Seasonality content is almost always raw monthly averages, which mostly rediscover market beta - the version-one April artefact is the canonical example. Using excess returns against the Nifty plus a t-test bar, ranking by t-statistic, and tagging each pattern with the real Indian calendar driver produces edges that survive a persistence test, including negative ones
6	**The investor profile actually changes the output**	Risk questionnaires in most tools produce a label and then have no further effect. Here appetite and horizon impose a hard equity ceiling, adjust it by horizon band, and reshape the defensive split, while the position size derives from the model's edge over the index and the stock's quality. The same stock genuinely produces different allocations for different investors
7	**Uncertainty is surfaced, not smoothed away**	Three individual votes with confidences, an explicit agreement ratio, the rule cross-check as a separate line, three separate importance charts, disagreement listed as a demerit, and significance stars on seasonal claims. The system is designed to tell the user when it does not know

What is not novel, stated plainly

The indicator mathematics is textbook. The model families are standard. The data source is free and retail-grade, with the survivorship and label-quality limitations that implies. The knowledge base is hand-curated and therefore as current as its last edit. The novelty is in the target design, the composition, the India specificity and the discipline about honesty - not in any single algorithm.

SECTION 15

Limitations and caveats

What the system cannot do, stated before anyone has to discover it

Area	Limitation	Consequence and any mitigation
Prediction semantics	OUTPERFORM does not mean the price rises	In a falling market it can mean losing money, just less than the index. Mitigated by stating it explicitly in the interface, the merits panel and the language-model prompt, but it remains the single most likely misinterpretation
Data source	Yahoo Finance is the only market-data source	Free, retail-grade, occasionally inconsistent in its field names, and three training tickers fail outright on symbol quirks. No corporate-action or survivorship handling. A single upstream change can affect everything
Model inputs	Fundamentals and sentiment are not model features	A deliberate honesty trade-off - they cannot be reconstructed historically from free data. It means the model is purely technical and price-derived, and business quality only enters through the composite score
Explainability	Feature importances are global, not per prediction	Every stock shows the same driver ranking. Per-prediction attribution such as SHAP values is the clearest next improvement
Backtest scope	The strategy backtester models no transaction costs, slippage, impact or Indian securities transaction tax	Reported returns are optimistic. This is standard for a first-pass backtester but must not be read as achievable performance
Seasonality	Roughly ten annual observations per sector-month	Per-edge out-of-sample samples are around four. Only the aggregate persistence read is meaningful; the harness says so itself
Knowledge base	The geopolitical base is hand-curated and static	It is only as current as its last edit and cannot detect a new conflict or an escalation on its own
Language model	Both language-model layers depend on an external API	Subject to quota, latency and model deprecation - a retired model identifier has already broken this once. Failures degrade to a message, never to a fabricated insight. The underlying client library is also deprecated in favour of a newer one
Performance	A cold analysis takes around 25 seconds	Driven by FinBERT loading and multiple sequential network fetches. Mitigated by disabling article-body scraping on the per-stock path, a thirty-minute sentiment cache, an hourly peer cache and a six-hour seasonal cache, but the analysers still run sequentially rather than concurrently
Deployment	Runs on the Flask development server with no authentication or rate limiting	Single-user local deployment only. Gunicorn is declared but not wired up
Testing	No automated test suite	Verification has been manual and end-to-end. This is the largest engineering gap in the repository

SCOPE STATEMENT

This is an educational analysis system. It produces no personalised financial advice, and both the portfolio and merits layers are written to be non-directive by construction - they describe who a stock suits and why, and never instruct anyone to buy or sell. Every language-model output closes on that caveat.

SECTION 16

Roadmap: what to build next

Ordered by the ratio of value to effort, not by ambition

Tier 1 - closes a gap that currently undermines existing claims

Item	What it involves	Why it matters most
Per-prediction explanations	SHAP values over the tree ensemble, surfaced per analysis alongside the existing global importances	The interface currently shows the same drivers for every stock, which undercuts the explainability the rest of the system works hard for. This is the single highest-value addition
An automated test suite	Unit tests on the feature builder, the scoring maths and the sanitiser; contract tests on each endpoint; a golden-file test on a known ticker	The largest engineering gap. Several past bugs - the inverted equity sizing, the stuck mount reference, the train/serve mismatch - were silent and would have been caught
Realistic backtest costs	Brokerage, securities transaction tax, stamp duty, slippage and impact in the backtester	Without them the reported strategy returns are not just optimistic but structurally misleading
Concurrent analysers	Run the six independent analysers and their network fetches in parallel	They are already isolated and have no interdependencies, so this is close to free and would cut the 25-second cold path substantially
Walk-forward validation	Replace the single time split with rolling retrain-and-test windows across the whole history	One cutoff date is one sample of the model's skill. Rolling windows show whether the edge is stable or an artefact of one regime

Tier 2 - extends the system along its existing grain

Item	What it involves	The reasoning
Multi-stock portfolio construction	Take several analysed stocks and build a covariance-aware portfolio - mean-variance or risk parity - instead of one stock against a defensive sleeve	The optimiser is single-name today. The predicted outperformance probabilities are exactly the expected-return inputs a portfolio optimiser needs, so the hard part is already computed
Cross-sectional ranking	Score the whole 150-ticker universe nightly and expose a ranked screen	The model already predicts relative performance, which is inherently a cross-sectional quantity. Ranking the universe is the natural expression of the target and turns the tool from a lookup into a screener
Live event detection for the War Room	Ingest news and event feeds, match against conflict escalation triggers - which are already encoded per conflict - and flag escalations	The knowledge base already contains explicit triggers. Nothing watches for them, so the module is reactive when it could be monitoring
Geopolitical signal backtest	Test whether the exposure factor has predicted returns around historical escalation dates	It is the most novel factor and the only one with no validation at all
Alternative fundamental source	A source with genuine historical depth, enabling fundamentals as real model features	This is the honest way to lift the deliberate limitation that fundamentals sit outside the model

Item	What it involves	The reasoning
Sector and index-level models	Apply the same relative-performance framing to sector indices against the Nifty	Would connect the seasonal module's rotation findings to the machine-learning stack, which are currently two separate stories about the same phenomenon
Persistence and watchlists	A database for analysis history, saved watchlists and prediction tracking	Every analysis is currently thrown away. Storing predictions with their outcomes would produce a live accuracy record - the most credible evidence the system could offer

Tier 3 - larger bets

Item	What it involves	Honest assessment
Sequence models	Temporal convolutional networks, gated recurrent units or a small transformer over price sequences instead of point-in-time engineered features	The engineered features already compress most of the accessible signal in daily OHLCV. Worth trying, but expect modest gains - the current features are not the binding constraint
Regime-conditional models	Separate models per volatility or trend regime, selected by the VIX z-score at inference	The VIX regime feature is in the model precisely because behaviour is regime-dependent. Explicitly conditioning on it is the logical next step and is cheap to test
Prediction-outcome feedback loop	Log every prediction, score it after twenty trading days, and report live rolling accuracy	Cheap to build and the most honest possible marketing. It also creates the dataset for periodic retraining
Options and derivatives layer	Implied volatility, put-call ratio and open-interest signals from NSE derivatives data	India has one of the world's most active derivatives markets, and options positioning is genuinely predictive information the system currently ignores entirely
Retrieval over filings	Extend the retrieval approach from geopolitics to annual reports, earnings transcripts and exchange disclosures	The War Room proves the pattern works. Applied to filings it would let the qualitative governance layer read primary sources instead of scanning news keywords
Production hardening	Gunicorn, authentication, rate limiting, structured logging, containerisation, a job queue for slow analyses	Necessary before any real user touches it, but strictly downstream of the correctness work in tier one

IF ONLY THREE THINGS GET BUILT

Per-prediction SHAP explanations, an automated test suite, and the prediction-outcome feedback loop. The first makes the existing intelligence legible per stock. The second protects everything already built from the class of silent bug that has bitten this project three times. The third replaces every claim in this document with measured, live evidence.

SECTION 17

Reference

API surface, repository layout and how to run it

API endpoints

Method and path	Purpose	Key inputs
`POST /api/analyze-stock`	The full multi-factor analysis: six factors, tri-model prediction, portfolio, merits, peers and the narrative insight	ticker, risk_appetite, horizon
`POST /api/profile`	The standalone questionnaire risk score, level and recommendations	goal, horizon, income, riskAppetite
`GET /api/conflicts`	The curated geopolitical watchlist for the War Room	none
`POST /api/war-analysis`	A grounded India-impact report from the watchlist or a free-text scenario	conflict_id or scenario_text
`GET /api/seasonal-analysis`	Ten-year market-relative sector seasonality, cached six hours	refresh (optional)
`GET /api/trading-strategies`	The available strategy catalogue	none
`POST /api/backtest-strategy`	Backtests a strategy and returns the full metric set	ticker, strategy_type, parameters, initial_capital, pair_ticker
`POST /api/strategy-signals`	The latest signal and position size for a strategy	ticker, strategy_type, account_value
`GET /api/health`	Liveness check for the frontend status dot	none

Running the system

Backend - from the backend directory, on the global Python 3.11

```
pip install -r requirements.txt
python app.py    # serves on http://localhost:5000
```

Frontend - from the frontend directory

```
npm install
npm start    # serves on http://localhost:3000, proxied to the API
```

Both at once, from the repository root

```
start.bat
```

Retraining the model - about five to eight minutes, requires network access

```
# delete models/stock_model.pkl first, then:
python -c "from models.stock_model import StockRiskModel; m=StockRiskModel(); m.train(); m.save()"
```

Validation harnesses

```
python -m models.backtest
python -m models.seasonal_backtest
```

Required environment variables in backend/.env

```
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-flash-latest   # optional override
```

Two operational notes

- The backend must run on the **global** Python 3.11 installation, not the project virtual environment - the latter has an inconsistent dependency set and a statsmodels version that fails on import.
- The trained model file is large and excluded from version control, so a fresh clone must train once before the machine-learning features work. The loader detects a missing or stale artefact and retrains automatically on first prediction.

Disclaimer. This document describes an educational analysis system. Neither the software nor this report constitutes financial advice, a recommendation to buy or sell any security, or a solicitation of any kind. All model outputs are probabilistic and relative to a benchmark index; past performance and backtested results do not indicate future returns.